



 Sep 27, 2026

Let's start with the core of backend development now!

Overview

In this task, you will implement a **secure user authentication system** using Node.js and Express.

You will build registration and login functionality, securely hash user passwords, issue and verify JWT tokens, and create authentication middleware to protect routes. You will also integrate email notifications using **Nodemailer**.

The goal is to strengthen your understanding of **authentication, authorization, JWTs, middleware, password security, and email services in Node.js**.

Goals

Database

- Use MongoDB Atlas with Mongoose (an ODM) to store and manage your application data.
- No boring SQL queries here, you can work with your database using JavaScript and Mongoose.

Postman

- Use Postman to test all your API endpoints. Create a collection for the task and organize your requests properly.
- Include screenshots of your API testing in your README.
- A clean, well-documented codebase with clear APIs is a sign of a good backend developer.

User Authentication

- Create Register and Login routes.
- Create a User model and securely hash passwords using `bcryptjs`.

MVC Structure

- Follow the MVC (Model–View–Controller) architecture strictly throughout the project.
- Keep models, controllers, routes, middleware, and other responsibilities properly separated.
- Avoid placing business logic directly inside route files.

JSON Web Token (JWT)

- Use JWT for authentication and authorization.
- Implement access **and** refresh **tokens**.
- Store JWT secrets and other sensitive information in `.env`

Authentication Middleware

- Create middleware to verify JWTs and protect routes.
- Attach the authenticated user's ID to the request object.

Mailing System

- Use **Nodemailer** to send a welcome email after registration.
- Send a login notification after successful login.

User Model

Create a User model using Mongoose with the following fields:

- name
- email
- password
- createdAt
- updatedAt

Use appropriate validation for the fields and configure timestamps for **createdAt** and **updatedAt**.

Model

User:

```
{
  "_id": ObjectId(),
  "name": String,
  "email": String,
  "password": String,
  "createdAt": Date,
  "updatedAt": Date
}
```

Useful Tips

-  Configure **email validation** directly in the Mongoose model.
- Add **password validation** to the Mongoose schema.
- Always hash passwords using **bcryptjs** and never store plain-text passwords.
- Never hardcode **JWT secrets** or **SMTP credentials**.
- Store sensitive configuration inside .env files.
- When storing decoded JWT data in the request object, keep only the **user's ID** instead of the entire user object. This keeps the request lightweight and avoids exposing unnecessary information.

Learning Resources



Reference Links

1. **MongoDB download** - <https://youtu.be/uFDLyMac3rc?si=JKdm6bd0B7ruo6pt>
2. **Bcrypt in NodeJS** - <https://www.youtube.com/watch?v=8imz7zRHkSM>
3. **Cookies and sessions in Nodejs:**
<https://youtu.be/kMErso06vHo?si=bjexkSEtNlRjZtet>
<https://youtu.be/YhIrMZa4zgQ?si=TePdIFZGnojaDjna>
Token Vs Session - <https://youtu.be/QzntvHz23tw?si=9XI1a4960Szh6RhY>
4. **MongoDB Crash Course** - <https://youtu.be/M1dKYQ7GsTg?si=1nLEzjGl3-i9otY0>
5. **JWT Auth in NodeJS** - https://www.youtube.com/watch?v=mGrVmEex6_g
6. **Express Middleware** - <https://www.youtube.com/watch?v=n2c0mf1sza4>
7. **Nodemailer** - <https://www.youtube.com/watch?v=n8fVC5UFRjA>



Documentation

1. **MongoDB** - <https://www.mongodb.com/docs/manual/core/document/>
2. **MVC** - <https://developer.mozilla.org/en-US/docs/Glossary/MVC>
3. **JWT:**
 - a. <https://www.jwt.io/introduction#what-is-json-web-token>
 - b. <https://www.geeksforgeeks.org/web-tech/json-web-token-jwt>
4. **Express Middleware** - <https://www.geeksforgeeks.org/node-js/explain-the-concept-of-middleware-in-nodejs/>
5. **Nodemailer** - <https://www.npmjs.com/package/nodemailer>